



PRODUCT VISION · HACKATHON BRIEF

Quiro AI

Edit your screen recording by talking to it.

Quiro turns a messy screen recording into a polished demo *while you talk to it*.
You record; you say what you want in plain language; the timeline edits itself.

OWNER

Bruno

STATUS

Proposal

UPDATED

Jun 17, 2026

VERSION

v1 Draft

WHAT'S INSIDE

- 1 The problem we're solving
- 2 Our insight — why Quiro
- 3 The vision — Quiro Director
- 4 The core — the Recording Brain
- 5 What we're adding
- 6 How it fits our architecture
- 7 Why we're different
- 8 Hackathon scope (MVP)
- 9 Tech approach
- 10 The demo
- 11 Risks & open questions
- 12 Suggested workstreams

TL;DR

Editing a screen recording takes far longer than recording it. Today Quiro gives users powerful **manual** tools (zoom, trim, speed, captions, annotations). We want to make Quiro **AI-native**: a conversational agent that **understands** the recording and **edits it for the user** by driving the same tools they'd use by hand.

Because every edit in Quiro is already structured data (regions), the AI doesn't need to "drive a UI" — it reads and writes regions directly. That is our unfair advantage and the reason we can ship something magical in a hackathon timeframe.

1 The problem we're solving

Recording is easy. Editing is the wall. For the people who make screen recordings — developers writing docs, founders making product demos, support teams, educators, creators — the work breaks down like this:

- **Editing takes 5–10× the recording time.** Placing zooms on every click, cutting dead air, removing "ums," speeding up boring stretches, writing captions — all manual.
- **Polish requires skill.** Knowing *when* to zoom, how to pace, where to cut is craft. Most people don't have it, so their recordings look amateur.
- **One recording, many needs.** The same demo needs to become a short, a README GIF, a captioned social clip, a help-doc walkthrough. Today that's N separate manual edits.
- **The tools don't understand the content.** Existing editors (including ours, today) don't know that at 0:42 the user clicked "Export," or that 0:10–0:18 is silence.

The result: people either ship rough, unpolished recordings, or they burn hours editing. Neither is good.

2 Our insight — why Quiro can win this

Two things make Quiro uniquely positioned:

- **Our editor is already a clean tool API.** Every edit — zoom / trim / clip / speed / audio / annotation — lives as a structured *region* in `window.tsx`. An AI agent doesn't have to simulate clicks; it calls a function that adds a region. **The editing surface is the tool surface.**
- **We already capture the signals an AI needs to understand a recording:** the **transcript** (whisper.cpp, already integrated), the **cursor & interaction stream** (our native cursor monitor), and the **frames themselves** (sampled for a vision model).

THE HEAD START

We are ~60% of the way to an AI-native editor before writing any AI code. The hackathon work is the glue, not the foundation.

3 The vision — "Quiro Director"

A conversational editing agent built into the editor. The user records, then says:

"Make this a punchy 60-second product demo. Zoom into every button I click, cut the dead air and my 'ums', add captions, and end on the pricing page."

...and the **timeline assembles itself in front of them** — zoom regions snap onto clicks, dead air collapses, captions appear. Then they refine conversationally: *"second zoom is too aggressive," "keep my intro,"* and the agent adjusts. The chat panel becomes a first-class surface that can **answer** questions, **edit** the video, **repurpose** it, and **teach** the app.

4 The core we're building — the "Recording Brain"

This is the real technical contribution and the thing everything else depends on. After a recording, we build **one structured semantic timeline** by fusing three signals:

Signal	Source	Gives us
Transcript	whisper.cpp (already integrated)	words + timestamps, silences, filler words, sentence/topic boundaries
Interaction stream	native cursor monitor (already integrated)	clicks, typing, scrolls, "user interacted here" moments
Scene understanding	sparse frame sampling → vision model	semantic labels: "settings open," "pricing page," "code editor visible"

Fuse these into a JSON timeline of **moments**, each with a time range, a description, and tags. **Every AI feature becomes a query over this structure.** Build it once; the rest is cheap.

5 What we're adding to the product (as a whole)

TIER 0 — FOUNDATION

Build first

- **The Recording Brain / semantic timeline.** Nothing else works without it.

TIER 1 — FLAGSHIP

The "wow"

- **Conversational edit agent ("Quiro Director").** Chat → tool calls → regions update live.
- **One-click auto-edit ("magic first draft").** Auto silence removal, filler-word cuts, auto-zoom on clicks, speed-ramps — the instant-gratification moment.

TIER 2 — MOAT

Depth that makes it real

- **Text-based editing.** Delete a sentence in the transcript → the video cuts that span.
- **Auto-repurpose / one-to-many export.** One recording → vertical short, README GIF, thread with timestamps.
- **AI voiceover & translation.** Clean up audio or re-voice the demo in another language with TTS + re-captioning.

TIER 3 — POLISH

Cheap generations off the Brain

- Auto chapters, title, thumbnail, callout annotations, and an in-editor "how do I..." copilot that can perform the action.

6 How it fits our architecture (no rewrites)

- **Tools = existing region handlers.** Expose `window.tsx` handlers as a tool schema. The agent is just another caller of the same state, so the performance-critical playback path is untouched.
- **API key lives in the main process.** Chat (renderer) → IPC → main calls Claude → tool calls return → renderer mutates regions. Adding the channel touches: handler, register file, `preload.ts`, and **both** `electron-env.d.ts` files.
- **Reuses what's there.** whisper, cursor monitor, region system, media servers. New surface = chat UI + agent loop + the Brain builder.
- **Doesn't break playback.** Agent edits are batch state updates, not high-frequency ones.

7 Why we're different

	Loom	Screen Studio	Quiro (this vision)
Auto-zoom	partial	mechanical	zooms based on what you did & said
Understands content	no	no	yes — transcript + interactions + vision
Edit by talking	no	no	yes — conversational agent
One-click first draft	limited	no	yes
Repurpose to many formats	no	no	yes

The category has *auto* features. **Nobody has an agent that understands the recording and edits it conversationally.** That's the wedge.

8 Hackathon scope (MVP)

Don't build all of Section 5. The winning slice:

- **Recording Brain** — transcript + click stream + a few vision frames → moments JSON.
- **Chat agent with 3 tools** — auto-zoom-on-click, silence/filler trim, captions.
- **One "magic first draft" button.**
- **A rehearsed 90-second live demo.**

Everything else becomes "...and it also does X" in Q&A. Transformation in the demo beats a long feature list.

9 Tech approach

- **Agent:** Claude API with **tool use** (function calling). Tools map 1:1 to region operations.
- **Models:** `claude-opus-4-8` as the planning brain; `claude-haiku-4-5` for cheap high-volume classification ("silence / filler / important?"). **Stream** the chat for a live feel.
- **Vision:** sample ~1 frame / few seconds for scene labels (used to build the Brain, not per-frame).
- **Security:** API key in main process only; never expose to renderer.

10 The demo (how we win the room)

- 1 Record a deliberately messy ~3-minute screencast live (pauses, "ums," idle time).
- 2 Type **one sentence** of intent.
- 3 Watch the timeline build itself: zooms snap to clicks, dead air collapses, captions appear.
- 4 One conversational refinement ("make the intro shorter").
- 5 Export → show the polished result.

"Editing a screen recording takes 10× the recording time. Quiro does it while you talk."

11 Risks & open questions

- **Latency.** Build the Brain async right after recording; use Haiku for bulk classification; stream the chat.
- **Edit quality / trust.** AI edits must be previewable and reversible — show the diff on the timeline; everything is an undoable region.
- **Vision cost/volume.** Keep frame sampling sparse; cache scene labels in the Brain.
- **Open:** Hackathon **timeframe**? Judging **criteria / theme**? Voice or text chat for v1? Online vs. any offline constraint?

12 Suggested workstreams (team split)

Workstream	Scope
A — Recording Brain	transcript + cursor/click capture + frame sampling → moments JSON
B — Agent & tools	tool schema over region handlers, IPC channel, agent loop, streaming
C — Chat UI & first draft	the surface users touch + the "magic first draft" button
D — Demo & pitch	script, recording, slides, narrative

This is a living doc — edit freely and bring questions to the team sync.